

Final Data Science Report: Video-on-Demand (VoD) Subscriber Churn Prediction

Course: Introduction to Data Science

Instructor: Prof. Yrupe Fresco

Author: iralai

Project: Customer Churn Predictor App Framework

1. Domain Context & Churn Label Justification

In a subscription Video-on-Demand (VoD) streaming environment, defining customer churn is an essential business decision rather than a simple software property. Unlike enterprise environments or developer networks (e.g., GitHub) where a user might be naturally silent for months, streaming media platforms rely heavily on a monthly recurring subscription interval.

Our Applied Churn Criteria

For this application, a subscriber is flagged as churned (1) if they have a prolonged window of physical system inactivity (greater than 30 days) or if their aggregated monthly watch consumption collapses (below 5 hours) while platform friction mounts.

Why a 30-Day Threshold is Scientifically Correct for VoD

Contract Cycle Alignment: Waiting for 90 or 180 days of inactivity before flagging churn would be financially catastrophic for a streaming service. By that time, multiple billing cycles would have failed, the customer habit loop would be completely broken, and active user acquisition costs would skyrocket.

Proactive Window: A 30-day window captures the behavioral drop-off exactly at the expiration of their monthly subscription boundary. This gives the marketing and product teams a high-leverage window to execute automated, pro-active retention campaigns before the user permanently leaves the platform.

Class Balance Design Decision

The dataset was deliberately forced to a **50/50 balanced distribution** between churned (1) and retained (0) subscribers by sampling `min(len(churned), len(retained))` records from each class before model training.

Why This Was Done

In real-world VoD streaming environments, natural churn rates typically fall between **5–20%** of the subscriber base. Training a classifier on this raw imbalanced distribution causes the model to default toward predicting the majority class (retained), achieving misleadingly high accuracy while completely failing to identify actual churners — the exact population we care about most.

By enforcing a balanced training set, the Random Forest classifier is forced to learn genuinely discriminating decision boundaries rather than exploiting class frequency as a shortcut.

Acknowledged Limitations

This approach does **not** reflect the true prior probability of churn in production. Specifically:

- **Precision will be artificially optimistic** during evaluation, since the model was never exposed to the realistic class ratio it will encounter in live inference.
- **Churn probability outputs** from `/predict` should be interpreted as relative risk scores rather than calibrated real-world probabilities.

Production Recommendation

In a deployed system, the preferred approach would be to preserve the natural class distribution and instead pass `class_weight="balanced"` to the `RandomForestClassifier`:

```
model = RandomForestClassifier(  
    random_state=42,  
    n_estimators=100,  
    class_weight="balanced"  
)
```

This allows the model to train on realistic data proportions while internally up-weighting the minority class during loss computation, producing better-calibrated probability outputs suitable for threshold-based business interventions.

2. Synthesized Feature Selection Analysis

To determine which user signals are highly indicative of subscription retention, we processed our 8 engineered behavioral features across all 4 required machine learning feature selection methods.

The Consolidated Evaluation Matrix

Our notebook executed an integrated pipeline yielding the following empirical system rankings:

Feature Name	Feature Type	Filter Rank	RFE Rank	DT Rank	RF Rank	Final Strategic Decision
days_since_last_login	Time-Based (Recency)	1	1	1	1	Keep
monthly_watch_hours	Aggregation (Volume)	2	2	2	2	Keep

Feature Name	Feature Type	Filter Rank	RFE Rank	DT Rank	RF Rank	Final Strategic Decision
preferred_content_meta	OMDb Metric	7	8	4	3	Keep
preferred_content_imdb	OMDb Metric	4	5	3	4	Keep
avg_completion_rate	Ratio	5	7	5	5	⚠ Optional
subscription_age_months	Time-Based (Tenure)	3	4	7	6	⚠ Optional
watchlist_size	Ratio	6	6	6	7	Drop
customer_support_tickets	Aggregation (Friction)	8	3	8	8	Drop / Segregate

3. Explaining Core Disagreements: Linear vs. Non-Linear Models

RFE Top 5 vs. Filter Top 5

The Wrapper method (RFE using Logistic Regression) selected these top 5 features in order:

1. `days_since_last_login`
2. `monthly_watch_hours`
3. `customer_support_tickets`
4. `subscription_age_months`
5. `preferred_content_imdb`

By contrast, the Filter method (absolute Pearson correlation) ranked its top 5 as:

1. `days_since_last_login`
2. `monthly_watch_hours`
3. `subscription_age_months`
4. `preferred_content_imdb`
5. `avg_completion_rate`

The two methods agree on the first two slots — `days_since_last_login` and `monthly_watch_hours` are universally strong churn signals. The critical difference is that **RFE swaps `avg_completion_rate` for `customer_support_tickets`** at position 3/5. The Filter method ranks `customer_support_tickets` dead last (8th), while RFE elevates it to 3rd. This disagreement is analytically valuable because it reveals a fundamental limitation of filter-based ranking.

Why This Disagreement Matters

The Filter method, a single Decision Tree, and the Random Forest ensemble completely ignored `customer_support_tickets` (ranking it 8th place). However, RFE prioritized it as the 3rd most critical feature.

The Mathematical Explanation

This striking disagreement occurs because RFE utilizes a linear model (Logistic Regression) as its base estimator, whereas Decision Trees and Random Forests construct non-linear splits.

In our dataset, customer support tickets represent an **interaction effect** rather than a straight linear driver. A high number of customer service tickets alone does not cause an active subscriber to churn; if they have exceptionally high watch hours, they love the platform's content enough to tolerate technical friction.

However, if their support tickets scale up while their monthly watch hours are low, a threshold breaks and they cancel. Linear engines like RFE evaluate features along a flat plane and miss this context, artificially inflating the weight of tickets to compensate. Tree models isolate this naturally via sequential splits (e.g., `IF hours < 10 AND tickets > 3`). This shows that non-linear tree ensembles provide a much more accurate reflection of true user behavior.

4. Model Performance Comparison: Original Features vs. PCA vs. Original + Network

To evaluate how different feature engineering strategies affect prediction quality, we compared Random Forest performance across three setups using 5-fold cross-validation:

Feature Setup	Accuracy	Precision	Recall	F1 Score
Original Features (10 vars)	0.920 (+/- 0.015)	0.918 (+/- 0.022)	0.921 (+/- 0.018)	0.919 (+/- 0.016)
PCA Components	0.885 (+/- 0.021)	0.881 (+/- 0.025)	0.887 (+/- 0.019)	0.884 (+/- 0.020)
Original + Network (12 vars)	0.931 (+/- 0.013)	0.929 (+/- 0.019)	0.933 (+/- 0.015)	0.931 (+/- 0.014)

Key Findings

Original features perform well — the 10 engineered behavioral signals already capture the core churn drivers. PCA dimensionality reduction lowers variance but sacrifices ~3.5% accuracy because linear compression loses the interaction effects that tree models exploit.

Network features provide a marginal lift — adding degree centrality and PageRank centrality improves F1 by ~1.2%, confirming that peer-communication topology captures residual churn signal not present in individual behavior alone. However, the gain is modest relative to the cost of constructing the graph at scale ($O(n^2)$ edges), suggesting network features are best reserved for offline batch scoring rather than real-time inference.

Conclusion: The original 10 engineered features strike the best accuracy-to-complexity ratio for a production REST API. Network features offer a small upside for offline churn analysis.

5. Operational Business Interventions (Retention Strategy)

Predictive modeling is only valuable if it drives business actions. By passing live telemetry inputs into our FastAPI `/predict` endpoint, we translate churn probabilities into automated customer retention workflows:

High Churn Risk Trigger (Probability ≥ 0.70)

Behavioral Profile: Driven by an increase in `days_since_last_login` paired with a drop in `avg_completion_rate`.

Action: The platform automatically triggers a **Hyper-Personalized Re-engagement Campaign**. This leverages the user's preferred OMDb metrics (`preferred_content_imdb / preferred_content_meta`) to dynamically display trailers for highly acclaimed movies in their favorite genres directly via email and push notifications.

Medium Risk / Friction Mitigation (Probability 0.40 - 0.69)

Behavioral Profile: Driven by an accumulation of `customer_support_tickets` combined with declining tenure.

Action: The account bypasses standard automated bots and is routed to a premium support queue to proactively resolve system friction before the billing month rolls over.

At-Risk Recommendations (/recommend Endpoint)

The `/recommend/{user_id}` endpoint extends the pipeline from prediction to retention. When a user's churn probability exceeds 0.5, the API returns personalized movie recommendations based on their `preferred_content_imdb` profile — high-IMDb-rated titles that align with their demonstrated taste. This closes the loop: the model identifies who is at risk, and the recommendation engine gives the business a concrete tool to re-engage them.

6. Strategic Ethical Dimensions

Deploying a probabilistic churn predictor introduces key ethical and business trade-offs that management must consider:

The Danger of System Gaming

If subscribers discover that the platform targets high-risk users with deeply discounted subscription rates or free premium months, healthy users will quickly learn to change their behavioral signals (e.g., intentionally avoiding logins for two weeks) to trigger artificial discounts. This can lead to a severe decline in Average Revenue Per User (ARPU).

Notification Fatigue and Intrusion

Users flagged as high-risk simply because they have a low `watchlist_size` might find continuous re-engagement notifications intrusive. Over-communicating with an already unengaged customer can annoy them and accelerate their decision to cancel.

7. PM Reflection: Acting on a Combined Predict + Recommend Pipeline

If I were a product manager at **Netflix** or **Spotify**, the predict + recommend pipeline would fundamentally alter how I approach retention.

Shift from Reactive to Proactive Retention

Currently, most platforms act on churn *after* it happens — a user cancels, triggering a "we miss you" email. This pipeline allows us to act *before* cancellation. The `/predict` endpoint identifies at-risk users, and the `/recommend` endpoint gives us a concrete intervention to deploy immediately. At Netflix, this means serving a curated row of highly-rated content on the homepage the moment a user's login frequency drops. At Spotify, it means surfacing a "Discover Weekly" playlist tailored to an artist the user hasn't listened to in 30 days.

The 0.5 Probability Threshold as a Business Lever

Choosing the threshold for triggering recommendations is a product decision, not a model decision. A low threshold (0.3) casts a wide net but risks annoying users who are still active. A high threshold (0.7) conserves intervention budget but misses some churners. As PM, I would A/B test threshold values against retention lift at 30/60/90 days — the optimal point is where the marginal cost of the recommendation (email send, homepage slot, push notification) is exceeded by the marginal revenue of the retained subscription.

Feature-Driven Product Roadmap

The feature selection analysis directly informs product priorities:

- **days_since_last_login** is the single strongest churn signal. This tells me the product should invest in re-engagement triggers — push notifications, email digests, and personalized content alerts — that fire proactively after 7/14/21 days of inactivity, not just at the 30-day churn boundary.
- **customer_support_tickets** interacts non-linearly with engagement. A high ticket count combined with low watch hours is a churn emergency, but an otherwise engaged user with many tickets is simply vocal. The product response should differ: the former gets a human support outreach with a retention offer; the latter gets a technical fix.

Ethical Guardrails

The PM must also ensure the system is not manipulative. Offering discounts only to predicted churners can create perverse incentives (users learn to "game" the model). A better approach is to frame the recommendation as a universal discovery feature — "because you watched X" — rather than a churn-contingent reward. This preserves trust while still delivering the retention benefit.

8. Final Conclusion & Model Selection

Based on the experimental synthesis across all 4 selection methods, we selected our top features and successfully serialized a Random Forest Classifier into `app/model.pkl`.

By containing this entire pipeline within `docker-compose`, we eliminate environment friction entirely. The system scales effortlessly, validates payloads strictly via Pydantic models, and delivers an end-to-end reproducible machine learning architecture ready for full production-grade deployment.

9. Github Repository Link

Link: <https://github.com/ednairala/movie-churn-app.git>